

Bot notificări apartamente Craiova

Verifică periodic OLX, Storia și Publi24 pentru anunțuri noi de apartamente de vânzare în Craiova și îți trimite un mesaj pe Telegram pentru fiecare anunț nou.

1. Creează botul de Telegram (2 minute)

1. Deschide Telegram și caută **@BotFather**.
2. Trimite-i `/newbot` , dă-i un nume și un username (trebuie să se termine în `_bot` , ex. `craiova_apart_bot`).
3. BotFather îți dă un **token** de forma `123456789:AAExxxxxxxxxxxxxxxxxxxxxxx` .
Reține-l — e `TELEGRAM_BOT_TOKEN` .
4. Trimite-i botului tău nou creat un mesaj oarecare (ex. `/start`) — altfel nu-ți poate trimite el notificări.
5. Află-ți `chat_id` : caută pe Telegram **@userinfobot** și trimite-i orice

mesaj; îți răspunde cu ID-ul tău numeric. Ți se
e `TELEGRAM_CHAT_ID`.

2. Rulează local, ca test (recomandat înainte de pasul 3)

```
cd craiova-apartment-bot
pip install -r requirements.txt

export TELEGRAM_BOT_TOKEN="tokenul-tau"
export TELEGRAM_CHAT_ID="chat-id-ul-tau"

python main.py --debug
```

Prima rulare doar "înregistrează" anunțurile
existente (nu trimite
notificări — altfel ai primi câteva sute deodată).
Rulează-l a doua oară
ca să vezi cum arată o notificare reală (dacă între
timp a apărut ceva nou)
sau șterge o linie din `seen_ids.json` ca să
simulezi un anunț "nou".

Dacă vezi `0 anunturi gasite` pentru un site,
structura paginii s-a
schimbat — vezi secțiunea **Depanare** mai jos.

3. Pune-l să ruleze automat, gratuit, pe GitHub Actions

Nu ai nevoie de server sau Raspberry Pi.

1. Creează un repo nou pe GitHub — **public** (repo-urile publice au minute nelimitate gratuite pe Actions; cele private au o cotă lunară limitată care s-ar epuiza repede la verificări din 10 în 10 minute).

Codul nu conține date personale — tokenul stă separat, în Secrets.

2. Încarcă toate fișierele din acest folder în repo.
3. În repo: **Settings** → **Secrets and variables** → **Actions** → **New repository secret**. Adaugă:
 - `TELEGRAM_BOT_TOKEN`
 - `TELEGRAM_CHAT_ID`
4. Mergi la tab-ul **Actions**, selectează workflow-ul „Verifica anunturi apartamente Craiova” și rulează-l manual o dată (`Run workflow`) — asta face prima rulare de "înregistrare", fără spam.
5. De acum încolo rulează singur, la fiecare ~10 minute.

3.5. Filtrul „doar de la proprietari”

E activat implicit (`ONLY_OWNERS = True` în `config.py`). Fiabilitatea diferă mult de la un site la altul:

- **OLX** — filtrare din conținut: exclude cardurile marcate cu eticheta „Firmă” (agențiile postează de pe conturi de tip firmă). Best-effort.
- **Storia** — la fel, best-effort: scriptul încearcă mai multe câmpuri probabile din datele interne ale site-ului (`isPrivateOwner` , `advertiserType` , nume de agenție). Rulează cu `--debug` și caută linia `[Storia] Chei disponibile...` ca să vezi exact ce date există, dacă vrei să-mi ceri un ajustaj.
- **Publi24** — nu are un filtru real la nivel de site (butonul „Persoane”/ „Companii” e doar vizual, verificat manual că nu are efect). Am implementat în schimb o detecție din textul anunțului: proprietarii scriu aproape mereu „Proprietar, vând...” sau „fără agenție”, iar agențiile scriu de obicei „<Nume firmă> vă propune/oferă spre vânzare”. E un tipar

real, verificat pe anunțuri de pe site, dar nu perfect — unele anunțuri de agenție formulate altfel (fără fraza tipică) pot trece nedetectate. Mai bine trec câteva anunțuri de agenție nedetectate, decât să rateze un anunț real de la un proprietar.

Dacă vrei să vezi din nou și anunțurile de la agenții pe OLX/Storia, pune

```
ONLY_OWNERS = False în config.py.
```

3.6. Dacă un site nu returnează niciun anunț

Am adăugat diagnosticare mai bună: workflow-ul rulează acum implicit cu

```
python main.py --debug
```

, deci în log vezi automat codul HTTP primit de la fiecare site și un fragment din răspuns dacă ceva nu e 200 OK.

Verifică în **Actions → (ultima rulare) → check →**

Ruleaza scriptul de

verificare. Cauze posibile dacă un site arată 0 `anunturi`:

- **HTTP 403 / pagină de verificare în fragmentul afișat** — site-ul a blocat cererea venită de pe serverele GitHub

(IP-uri cloud, des
blocate de sisteme anti-bot). Nu e ceva ce poți
regla din `config.py` ;
ar avea nevoie de un proxy sau de pachetul
`cloudscraper` . Trimite-mi
fragmentul din log și văd ce se poate face.

- **HTTP 200 dar tot 0 rezultate** — structura HTML
s-a schimbat.
Trimite-mi mesajul de eroare/output și ajustez
selectoarele.

4. Ajustează filtrele

Deschide `config.py` :

- `KEYWORDS_EXCLUDE` — cuvinte care, dacă apar în
titlu, exclud anunțul.
- `SOURCES` — poți dezactiva un site punând
`False` .
- `SEARCH_URLS` — dacă vrei alte filtre (preț, zonă,
camere), caută-le
direct pe site, apoi copiază URL-ul rezultat aici.

Depanare

- **Un site returnează mereu 0 rezultate:** sau
structura HTML s-a
schimbat, sau site-ul a blocat cererea. Rulează
`python main.py --debug`

și citește mesajele — îți spun exact unde s-a oprit extracția. Deschide site-ul în browser cu "View Page Source" și compară cu ce caută `scrapers.py`.

- **OLX și Storia au protecție anti-bot** mai agresivă decât Publi24.
Dacă primești erori HTTP 403 constant (nu ocazional), ia în calcul înlocuirea `requests` cu pachetul `cloudscraper` pentru acele funcții, sau mărește intervalul dintre verificări.
- **Imobiliare.ro nu e inclus** — nu am putut confirma URL-ul exact de căutare din acest chat. Dacă vrei să-l adaugi, trimite-mi link-ul de căutare filtrat pe Craiova de pe imobiliare.ro și îți scriu funcția de scraping pentru el.
- **git push eșuează în workflow**: verifică la Settings → Actions → General → Workflow permissions, că e bifat "Read and write permissions".

Notă etică

Scriptul verifică o dată la ~10 minute, ceea ce e un ritm rezonabil și

nu pune presiune pe servere. E gândit pentru uz personal (căutarea unei locuințe), nu pentru republicarea sau revânzarea datelor extrase.